3.5 Calling parent methods

Chapter 19 calls reusing a parent's implementation while adding child behavior **augmentation**. It is still an override. `parent::methodName()` explicitly invokes the inherited implementation on the current object; it does not create another object.

Extend a returned value

Run this complete example in `3-5-parent-methods.php`:

```php
<?php
declare(strict_types=1);

class Food
{
    public function __construct(protected string $name)
    {
        if (trim($name) === '') {
            throw new InvalidArgumentException('Name cannot be empty.');
        }
    }

    public function __toString(): string
    {
        return 'Food: ' . $this->name;
    }
}

class Dessert extends Food
{
    private int $calories;

    public function __construct(string $name, int $calories)
    {
        parent::__construct($name);

        if ($calories < 0) {
            throw new InvalidArgumentException('Calories cannot be negative.');
        }

        $this->calories = $calories;
    }

    #[Override]
    public function __toString(): string
    {
        $description = parent::__toString();

        return $description . ' (' . $this->calories . ' calories)';
    }
```

```
}

echo new Dessert('Berry tart', 240), PHP_EOL;
```

Output: `Food: Berry tart (240 calories)`.

Trace: The child constructor initializes the parent's name through the parent constructor, then initializes calories. The string method asks the parent to build the common description, then appends information. If the parent changes its formatting, the child benefits automatically.

Calling `$this->__toString()` from inside this override would call the override again and recurse. Calling `parent::__toString()` selects the inherited body. Always preserve the returned value when the caller needs it.

Decide whether to call the parent

Read the parent's contract and implementation first. Identify its inputs, output, state changes, validations, and side effects. Then ask whether your method is extending that work or supplying a complete replacement.

Situation	Placement	Reason
Child needs a value or initialized state produced by the parent	Parent first, then child work	The child depends on that result
Child must normalize or enrich the input used by the parent	Child preparation, then parent	The parent should receive the prepared input
Child adds behavior around a shared operation	Before and after one parent call	Separate preparation, common work, and finishing work
Child supplies the complete promised behavior	No parent call	Running both implementations may duplicate work
Parent method is abstract	No parent call	There is no implementation to invoke

Skipping the parent is valid only if the child's implementation still satisfies the contract. Do not skip required validation, persistence, or accounting merely because PHP permits it. Avoid running side effects twice. `#[Override]` does not enforce parent calls.

Demonstrate ordering with observable output

Run this independent example in `3-5-call-order.php`:

```
<?php
declare(strict_types=1);

class LabelMaker
{
    public function make(string $text): string
    {
        echo 'Parent received: ', $text, PHP_EOL;
```

```
        return '[' . $text . ']';
    }
}

class TrimmedLabelMaker extends LabelMaker
{
    #[Override]
    public function make(string $text): string
    {
        echo 'Child prepares', PHP_EOL;
        $text = trim($text);
        $label = parent::make($text);
        echo 'Child finishes', PHP_EOL;

        return $label . '!';
    }
}

echo (new TrimmedLabelMaker())->make('  Sale  '), PHP_EOL;
```

```
Child prepares
Parent received: Sale
Child finishes
[Sale]!
```

Move `trim()` below the parent call and rerun. The parent now receives the original spaces; changing the local input afterward cannot change the string the parent already returned. Restore the original order.

An order-saving method should normalize input before saving, and send a success notification only after a successful save. “Parent last” is not a universal cleanup rule. If cleanup must happen even when an exception occurs, later exception-handling lessons will use `try/finally`; code after a failed call does not execute automatically.

For a conditional parent call, define what the skipped branch returns and which guarantees still hold. For example, returning a cached calculation may be appropriate for a pure calculation, but skipping a save while reporting success breaks the method's promise.

Constructors require special attention

If the child declares **no constructor**, an accessible parent constructor is inherited. If the child declares **its own constructor**, PHP does not automatically invoke the parent constructor. Call `parent::__construct(...)` when its initialization is required. See the PHP constructor reference.

Temporarily remove the parent constructor call in the Dessert example. Construction completes, but converting the object to a string fails when it reads the uninitialized typed name property. Restore the call.

Parent-first works in this example, but preparation may need to happen first: normalize the input, then pass it to the parent. Be especially careful if a parent constructor calls an overridable method: it can dispatch into

child code before child state is initialized. Prefer constructor logic that does not depend on overridable behavior.

Constructors may take different parameters from their parents, and cannot use Override. There is no blanket PHP rule that parent must always be first or always last.